04_StaDa_API_Validation

September 23, 2026

1 Phase 3A: Automatisierte Referenzdatenbeschaffung via Deutsche Bahn StaDa-API

Dieses Hilfsnotebook dient der informationstechnischen Anbindung und dem Datenabruf aus der offiziellen StationData-Schnittstelle (StaDa-API v2) der Deutschen Bahn AG. Die vorangegangenen Untersuchungen in Phase 2 haben systematische Datenlücken bei kleineren Haltepunkten sowie Unregelmäßigkeiten bei der Erfassung von Großknotenpunkten (ID-Fragmentierungs-Hypothese) offengelegt.

Ziel dieses eigenständigen Moduls ist es, einen tagesaktuellen, offiziellen Gesamtkatalog aller deutschen Bahnhöfe direkt von den Servern des Infrastaturbetreibers zu beziehen. Die relevanten Attribute werden strukturiert extrahiert und zur Gewährleistung der Typentreue sowie zur verlustfreien Erhaltung verschachtelter Array-Strukturen in ein speicherschonendes Parquet-Format überführt.

1.1 Infrastrukturvoraussetzungen und Sicherheitskonzept der API-Schnittstelle

Für die erfolgreiche Ausführung des automatisierten Datenabrufs müssen zwei grundlegende technische Voraussetzungen erfüllt sein:

1. **Registrierung im DB-Entwicklerportal:** Der Zugriff auf die StaDa-API erfordert eine aktive Registrierung auf dem offiziellen API-Portal der Deutschen Bahn (`https://deutschebahn.com`). Nach dem Abonnement des entsprechenden API-Produkts (*StationData v2*) werden die systemischen Authentifizierungsschlüssel generiert.
2. **Kapselung der Credentials über Umgebungsvariablen:** Um kritische Sicherheitsrisiken und das versehentliche Veröffentlichen von Zugangsdaten zu verhindern, wird eine strikte Trennung von Programmcode und Geheimnissen implementiert. Die privaten Schlüssel (*Client-ID* und *Client-Secret*) werden lokal in einer nicht-öffentlichen Konfigurationsdatei mit dem Namen `.env` in der Projektwurzel hinterlegt.

   Für die Projektabgabe und die Reproduzierbarkeit durch die Prüfungskommission wird im Hauptverzeichnis eine Schablone namens `.env.example` bereitgestellt. Diese dokumentiert die erforderliche Variablenstruktur, enthält jedoch keine aktiven Zugangsdaten. Das Notebook lädt diese Werte zur Laufzeit über das `python-dotenv`-Framework direkt in den flüchtigen Arbeitsspeicher.

[1]:
```
import os
import json
import requests
```

```
import time
from pathlib import Path
import pandas as pd

# Automatisierte Installation fehlender Bibliotheken
try:
    from dotenv import load_workbook
except ImportError:
    !python.exe -m pip install python-dotenv requests pyarrow
    from dotenv import load_dotenv

from dotenv import load_dotenv

# Einbinden der zentralen Projekt-Infrastruktur
from project_config import (
    PROJECT_ROOT,
    TMP_DIR,
    STADA_API_RESULTS,
    ENV_FILE_PATH,
    get_project_relative_path
)

# Laden der API-Schlüssel aus der lokalen .env-Datei

if ENV_FILE_PATH.exists():
    load_dotenv(dotenv_path=ENV_FILE_PATH)
    print("ERFOLG: Umgebungsvariablen aus .env-Datei erfolgreich geladen.")
else:
    print("WARNUNG: Keine .env-Datei in der Projektwurzel gefunden. API-Aufruf␣
 ↪wird fehlerhaft sein.")

# Definition des temporären Pfads für die JSON-Rohdaten laut Systemkonzept
RAW_STADA_JSON_PATH = TMP_DIR / "stada_stations_raw.json"
```

```
Requirement already satisfied: python-dotenv in C:\0_DA\Python312\Lib\site-
packages (1.2.3)
Requirement already satisfied: requests in C:\0_DA\Python312\Lib\site-packages
(2.34.2)
Requirement already satisfied: pyarrow in C:\0_DA\Python312\Lib\site-packages
(25.0.1)
Requirement already satisfied: charset_normalizer<4,>=2 in
C:\0_DA\Python312\Lib\site-packages (from requests) (3.5.1)
Requirement already satisfied: idna<4,>=2.5 in C:\0_DA\Python312\Lib\site-
packages (from requests) (3.19)
Requirement already satisfied: urllib3<3,>=1.26 in C:\0_DA\Python312\Lib\site-
packages (from requests) (1.26.20)
Requirement already satisfied: certifi>=2023.5.7 in C:\0_DA\Python312\Lib\site-
packages (from requests) (2026.7.22)
```

```
ERFOLG: Umgebungsvariablen aus .env-Datei erfolgreich geladen.
```

1.2 REST-API-Abruf und Granulare Datenextraktion

Nachfolgend wird die REST-Schnittstelle der Deutschen Bahn operativ abgefragt. Der Request authentifiziert sich über die geladenen Header-Felder. Nach erfolgreichem Empfang der Rohdaten wird die JSON-Struktur geparst.

Die Transformation extrahiert selektiv die für die Netzwertvalidierung relevanten Attribute. Hierzu zählen geografische Zuordnungen, administrative Betriebsklassen (Bahnhofskategorien) sowie die komplexen, verschachtelten Listenstrukturen der assoziierten EVA-Nummern. Diese extrahierte Teilmenge wird anschließend zur performanten Weiterverarbeitung im Hauptnotebook gesichert.

[2]:
```
client_id = os.getenv("DB_API_CLIENT_ID")
client_secret = os.getenv("DB_API_CLIENT_SECRET")

api_url = "https://apis.deutschebahn.com/db-api-marketplace/apis/station-data/
 ↪v2/stations"
headers = {
    "DB-Client-Id": client_id,
    "DB-Api-Key": client_secret,
    "accept": "application/json"
}
```

[3]:
```
print(f"Sende GET-Request an die StaDa-API: {api_url}")
start_time = time.perf_counter()

try:
    response = requests.get(api_url, headers=headers, timeout=30)
    response.raise_for_status()
    daten_komplett = response.json()

    # Temporäre Sicherung der vollständigen Roh-JSON im tmp/-Verzeichnis
    with open(RAW_STADA_JSON_PATH, "w", encoding="utf-8") as raw_file:
        json.dump(daten_komplett, raw_file, ensure_ascii=False)
    print(f"Rohdaten erfolgreich im temporären Verzeichnis gesichert.")

except Exception as e:
    print(f"API-Abruf fehlgeschlagen: {e}")
    print("Nutze Fallback: Versuche existierende temporäre JSON-Datei zu laden..
 ↪.")
    with open(RAW_STADA_JSON_PATH, "r", encoding="utf-8") as raw_file:
        daten_komplett = json.load(raw_file)
```

```
Sende GET-Request an die StaDa-API: https://apis.deutschebahn.com/db-api-
marketplace/apis/station-data/v2/stations
Rohdaten erfolgreich im temporären Verzeichnis gesichert.
```

[4]:
```
alle_bahnhoefe = daten_komplett.get("result", [])
```

[5]:
```
flache_bahnhofs_liste = []

print(f"Metadaten-Prüfung: {len(alle_bahnhoefe)} Bahnhöfe im API-Katalog␣
 ↪identifiziert. Starte Denormalisierung...")

for bhf in alle_bahnhoefe:
    adresse = bhf.get("mailingAddress", {})
    prod_line = bhf.get("productLine", {})
    eva_liste = bhf.get("evaNumbers", [])

    # Extraktion der Produktdaten
    bahnhofs_typ = prod_line.get("productLine")
    netz_segment = prod_line.get("segment")

    # Da ein Bahnhof mehrere EVA-Nummern besitzen kann, wird für jede ID eine␣
 ↪flache Zeile erzeugt
    for eva_eintrag in eva_liste:
        eva_nummer = eva_eintrag.get("number")
        is_main_id = eva_eintrag.get("isMain", False)

        # Geografische Koordinaten aus dem GeoJSON-Punkt extrahieren
        geo_daten = eva_eintrag.get("geographicCoordinates", {})
        koordinaten = geo_daten.get("coordinates", [None, None])

        # GeoJSON-Standard: Index 0 ist Längengrad (Long), Index 1 ist␣
 ↪Breitengrad (Lat)
        long_wert = koordinaten[0] if len(koordinaten) > 0 else None
        lat_wert = koordinaten[1] if len(koordinaten) > 1 else None

        # Erstellung des flachen, relationalen Datensatzes
        flacher_bahnhof = {
            "api_station_name": bhf.get("name"),
            "api_city": adresse.get("city"),
            "api_zipcode": adresse.get("zipcode"),
            "api_federal_state": bhf.get("federalState"),
            "api_category": bhf.get("category"),
            "api_eva_nr": eva_nummer,
            "api_is_main_eva": is_main_id,
            "api_long": long_wert,
            "api_lat": lat_wert,
            "api_product_line": bahnhofs_typ,
            "api_segment": netz_segment
        }
        flache_bahnhofs_liste.append(flacher_bahnhof)
```

```
Metadaten-Prüfung: 5408 Bahnhöfe im API-Katalog identifiziert. Starte
Denormalisierung…
```

[6]:
```
df_stada_flat = pd.DataFrame(flache_bahnhofs_liste)
```

[7]:
```
df_muc = df_stada_flat[df_stada_flat.api_station_name == "München Hbf"]
```

[8]:
```
df_muc
```

[8]:

	api_station_name	api_city	api_zipcode	api_federal_state	api_category	api_eva_nr	api_is_main_eva	api_long	api_lat	api_product_line	api_segment
3094	München Hbf	München	80335	Bayern	1	8000261	True	11.558335	48.140232	Metropolbahnhof	Metropolbahnhof
3095	München Hbf	München	80335	Bayern	1	8098261	False	11.556708	48.141529	Metropolbahnhof	Metropolbahnhof
3096	München Hbf	München	80335	Bayern	1	8098262	False	11.554814	48.139930	Metropolbahnhof	Metropolbahnhof
3097	München Hbf	München	80335	Bayern	1	8098263	False	11.560493	48.141172	Metropolbahnhof	Metropolbahnhof

[9]:
```
# Typbereinigung zur Gewährleistung der mathematischen Vergleichbarkeit im Hauptnotebook
df_stada_flat["api_eva_nr"] = df_stada_flat["api_eva_nr"].astype("Int64")
df_stada_flat["api_category"] = df_stada_flat["api_category"].astype("Int64")
df_stada_flat["api_long"] = df_stada_flat["api_long"].astype("float64")
df_stada_flat["api_lat"] = df_stada_flat["api_lat"].astype("float64")
df_stada_flat["api_zipcode"] = (
    df_stada_flat["api_zipcode"]
    .astype("string")
    .str.strip()
    .replace("", pd.NA)
    .astype("Int64")
)
```

[10]:
```
df_stada_flat.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5461 entries, 0 to 5460
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   api_station_name   5461 non-null   str
 1   api_city           5461 non-null   str
 2   api_zipcode        5461 non-null   Int64
```

```
 3   api_federal_state  5461 non-null   str
 4   api_category       5461 non-null   Int64
 5   api_eva_nr         5461 non-null   Int64
 6   api_is_main_eva    5461 non-null   bool
 7   api_long           5461 non-null   float64
 8   api_lat            5461 non-null   float64
 9   api_product_line   5448 non-null   str
 10  api_segment        5448 non-null   str
dtypes: Int64(3), bool(1), float64(2), str(5)
memory usage: 831.1 KB
```

[11]:
```
problematische_werte = df_stada_flat[
    df_stada_flat.isna()
]
```

[12]:
```
display(problematische_werte.count())
```

```
api_station_name     0
api_city             0
api_zipcode          0
api_federal_state    0
api_category         0
api_eva_nr           0
api_is_main_eva      0
api_long             0
api_lat              0
api_product_line     0
api_segment          0
dtype: int64
```

[13]:
```
OUTPUT_RELATIVE_PARQUET = get_project_relative_path(STADA_API_RESULTS)
print(f"Sichere denormalisierte Referenzdaten unter (relativ):␣
 ↪{OUTPUT_RELATIVE_PARQUET}")

# Verlustfreier Export als saubere Parquet-Datei
df_stada_flat.to_parquet(OUTPUT_RELATIVE_PARQUET, index=False)

duration = time.perf_counter() - start_time
print("\n" + "="*70)
print("ERFOLG: Phase 3A (Datenbeschaffung & Denormalisierung) erfolgreich␣
 ↪abgeschlossen!")
print(f"Dimensionen der flachen Referenz-Tabelle: {df_stada_flat.shape}")
print(f"Verarbeitungszeit: {duration:.2f} Sekunden")
print("="*70)
```

```
Sichere denormalisierte Referenzdaten unter (relativ):
outputs/04_results.parquet

======================================================================
```

```
ERFOLG: Phase 3A (Datenbeschaffung & Denormalisierung) erfolgreich
abgeschlossen!
Dimensionen der flachen Referenz-Tabelle: (5461, 11)
Verarbeitungszeit: 7.90 Sekunden
======================================================================
```

[ ]: